



Proyecto Final

”Ruta Optima Entre Ciudades Mediante Grafos
Ponderados”

Carrera: Ing. Civil Informática

Asignatura: Matemática Discreta

Unidad: Teoría de Grafos

Profesor: Eliott Mardones

INTEGRANTES:

Benjamin Pedraza

Annan John

Vicente Huilcaman

Matias Espinoza

Universidad Católica, Temuco
Junio 2026

1. Introducción

El presente informe técnico demuestra, también explica el conocimiento adquirido y aplicado en la temática de grafos mediante la creación de un software que optimiza rutas entre un mínimo de 15 ciudades con 20 conexiones entre si representando la relación nodo-arista vista en grafos.

1.1 Ciudades Seleccionadas:

Las ciudades seleccionadas para el diseño de este programa son: “Madrid, Valladolid, Oviedo, Gijón, Santander, Bilbao, Vitoria, Zaragoza, Barcelona, Valencia, Alicante, Murcia, Granada, Córdoba, Sevilla, La Coruña, Vigo, San Sebastián, Málaga, Lisboa, Porto”.

1.2 Ponderacion De Grafo

El peso en este grafo ponderado es la distancia entre ciudad a ciudad, lo que permite poder solucionar de la mejor manera el problema principal; la ruta optima entre una ciudad de origen y otra de destino, mediante la toma de decisiones lógicas y eficientes.

Distancias Obtenidas en:
es.distance.to

2. Formulación Matemática

Para resolver el problema planteado de la mejor manera, utilizamos un modelo matemático el cual es una representación abstracta de relaciones y redes empleada en la teoría de grafos. A continuación, definiremos los componentes del modelo y lo que representa.

2.1 Definición formal de un grafo ponderado:

$$G = (V, E, w)$$

“ V ” Corresponde al conjunto de vértices o nodos que representan las ciudades elegidas en este proyecto.

“ E ” Corresponde al conjunto de aristas que representa las conexiones, rutas, carreteras, caminos, etc. Entre las ciudades elegidas.

“ w ” Corresponde al peso o valor numérico real y positivo asignado a cada arista (conexión entre ciudades) representado en Kilómetros (KM).

Conjunto de vértices (V):

El conjunto “ V ” está compuesto por todas las ciudades analizadas para este proyecto, siendo cada una distinta de otra.

El conjunto V se define como:

$$V(G) = \{\text{Madrid, Barcelona, Sevilla, Valencia, Zaragoza...Porto}\}$$

Cada elemento individual del conjunto “ V ” se denota como:

$$v_i \in V$$

Conjunto de Aristas (E):

Las aristas representan conexiones entre pares de vértices. Si una arista conecta los vértices u y v , puede escribirse como u, v

$$E(G) = \{\{\text{Madrid, Valencia}\}, \{\text{valladolid, vitoria}\}, \{\text{vigo, porto}\}, \dots\}$$

2.2 Función de peso w :

$$w : E \rightarrow r$$

Donde se le asigna un valor numérico a cada arista que representa la distancia en kilómetros (km) con las demás:

$$w(u, v) = d(u, v)$$

3. Desarrollo Del Software

3.1 Tipo de Grafo Utilizado:

En el desarrollo de este software utilizamos un grafo ponderado (cada arista o conexión tiene un peso o valor designado), conexo (todas las ciudades están conectadas de manera directa o indirecta) y no dirigido (la relación entre ciudades es bidireccional o sin dirección definida).

Significado de ruta óptima:

La ruta óptima en este software significa la menor distancia que se debe recorrer para llegar desde un origen a un destino, no la que menos ciudades recorre en el camino.

3.2 Elección Metodológica:

Para resolver el problema planteado de rutas óptimas entre ciudades se empleó el Algoritmo de Dijkstra usando el lenguaje de programación “Python”.

Se eligió este algoritmo porque es uno de los más eficientes y utilizados para este tipo de problemas, una de las características más útiles de este algoritmo es que al ser todas las distancias positivas entre ciudades el Algoritmo de Dijkstra se encarga de encontrar la ruta óptima de manera exacta y precisa (la menor suma de kilómetros entre la ciudad de origen y de destino).

También se podría haber seleccionado por ejemplo el Algoritmo de Kruskal pero no se eligió debido a que la función o el desarrollo del algoritmo se basa en “encontrar el árbol de expansión mínimo” (MST en inglés), esto significa que se conectan todos los vértices de un grafo usando el menor peso posible, cosa que no estamos intentando resolver sino la ruta más corta entre la ciudad de origen y destino, no conectarlo todo para usar solo una parte.

3.3 Funcionamiento general del software:

Para que el software pueda entender e interpretar “el mapa de las ciudades”, los datos se guardaron en un diccionario de Python llamado “grafo_ciudades” para poder generar la conocida “lista de adyacencia”.

El diccionario toma una ciudad de origen y lo asocia con una lista de sus “ciudades vecinas” con los kilómetros o distancia para llegar a ellas. Esta estructura es ideal ya que permite almacenar muchas ciudades sin tener “conexiones vacías” sino que solo “conexiones reales” a las cuales después se les consulta sus distancias para poder calcular la ruta de forma más directa.

4. Módulos y estructura de código

4.1 Implementación del Software

El software fue desarrollado en Python utilizando un diseño modular, por lo cual la estructura del código se dividió en 3 componentes principales para facilitar la lectura y el orden.

4.1.1. Grafo principal o Base de datos (Info_Grafos.py):

Este archivo contiene el diccionario con las ciudades seleccionadas, donde se almacena de manera ordenada y funcional los kilómetros y las conexiones entre ciudades

4.1.2. Lógica del Algoritmo (Algoritmo_Dijkstra.py):

Este archivo contiene la función matemática y logística de Dijkstra, la cual se encarga de recibir “el mapa de ciudades”, origen y destino para procesar todos estos datos mediante una cola de prioridad para así tener la ruta actual con los kilómetros totales.

4.1.3. Interfaz Gráfica (interfaz.py):

Es el “programa principal” debido que al ejecutarlo se acciona todo el proceso anterior, este archivo genera una interfaz gráfica creada utilizando la librería “Tkinter” y “Customtkinter” las cuales se complementan con las librerías “Matplotlib” y “NetworkX” encargadas de graficar.

5. Paso a Paso Del Software

Cuando el usuario interactúa o ejecuta el programa Main (interfaz.py) el sistema ejecuta una serie de pasos:

5.1 Captura y Limpieza de datos:

El software “toma” los nombres de las ciudades ingresada por el usuario, las transforma a minúsculas y elimina los espacios en blanco para evitar un error de escritura.

```
def ejecutar_dijkstra():
    inicio = entry_inicio.get().lower().strip()
    destino = entry_destino.get().lower().strip()
```

5.2 Validación de Ciudades:

Verifica si ambas ciudades que ingresó el usuario existen en la “base de datos” y si no existen muestra un error en la pantalla.

```
if inicio not in Info_Grafos.grafo_ciudades or destino not in Info_Grafos.grafo_ciudades:
    resultado_label.configure(
        text="Una o ambas ciudades no existen.\nPor favor, ingrese ciudades válidas.",
        text_color="red")
    return
```

5.3 Procesamiento y Cálculo:

Si el paso anterior se cumplió bien y las ciudades ingresadas son válidas, llama al módulo exclusivo de Dijkstra para calcular el camino óptimo.

```
camino, distancia = Algoritmo_dijkstra.dijkstra(Info_Grafos.grafo_ciudades, inicio, destino)

if camino:
    resultado_label.configure(
        text=f"Camino más corto de {inicio} a {destino}: \n(' -> '.join(camino))\nDistancia: {distancia} KM",
        text_color="green")
    mostrar_grafo(camino) # muestra el grafo con el camino resaltado en rojo
else:
    resultado_label.configure(
        text=f"No hay camino entre {inicio} y {destino}.",
        text_color="red")

def construir_grafo():
    G = nx.Graph()
    for nodo, vecinos in Info_Grafos.grafo_ciudades.items():
        for peso, vecino in vecinos:
            G.add_edge(nodo, vecino, weight=peso)
    return G

def mostrar_grafo(camino):
    plt.close('all')
    G = construir_grafo()

    pos = nx.spring_layout(G, seed=42, k=0.5)
    plt.figure(figsize=(10, 8))

    nx.draw(G, pos, with_labels=True, node_color='lightblue', node_size=1000, font_size=8)

    labels = nx.get_edge_attributes(G, 'weight')
    nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)

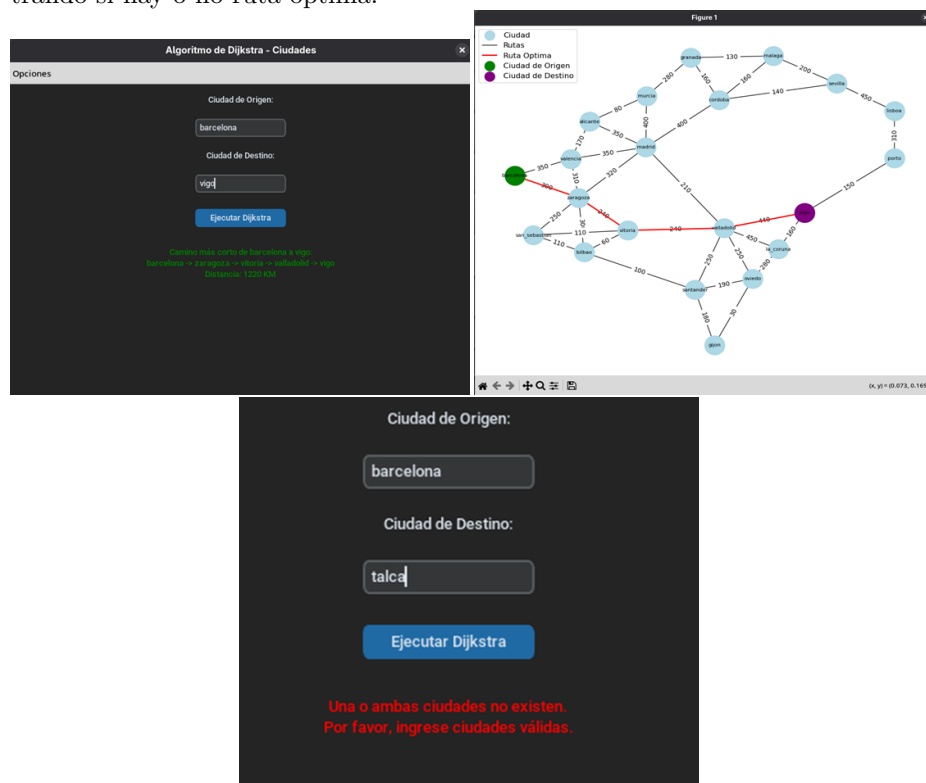
    edges_camino = [(camino[i], camino[i+1]) for i in range(len(camino)-1)]
    nx.draw_networkx_edges(G, pos, edgelist=edges_camino, edge_color='red', width=2)

    nx.draw_networkx_nodes(G, pos, nodelist=[camino[0]], node_color='green', node_size=1000)
    nx.draw_networkx_nodes(G, pos, nodelist=[camino[-1]], node_color='purple', node_size=1000)
```

6. RESULTADOS:

Para demostrar el correcto funcionamiento del software desarrollado, se realizaron pruebas en las cuales se ingresan ciudades de origen y destino en la interfaz gráfica.

A continuación, se mostrarán los resultados tanto del mapa como el texto mostrando si hay o no ruta óptima:



6.1 Conclusión:

Este proyecto nos permitió aplicar de manera práctica los conocimientos teóricos aprendidos en la asignatura “Matemática Discreta” para poder resolver un problema real de transporte y la resolución de la ruta óptima entre ciudades. A partir de los resultados obtenidos destacamos las siguientes conclusiones claves:

1. **La efectividad del modelo:**

Mediante la creación del grafo ponderado, conexo y no dirigido podemos comprobar que el modelado de “el mapa de ciudades” es fiel a la realidad demostrando la efectividad de este software.

2. **Eficiencia en el Software:**

Se demostró que al combinar el Algoritmo de Dijkstra con una cola de prioridad se cumple de manera satisfactoria el problema a resolver.

3. **Uso de la interfaz:**

La interfaz creada mediante “Tkinter” y “Customtkinter” demuestra que se pueden combinar de excelente manera con librerías de visualización gráfica (NetworkX y Matplotlib) siendo igual de eficiente que intuitivo.